

Comsats University Islamabad (Attock Campus)



Assignment no.03

Submitted By: Zoya Fiaz

Reg no: Fa24-Bse-022

Submitted TO : Mrs.Ambreen Gul

Date:4thMay 2026

Course: Information Security Lab

Question no.01:

TASK 1 — Basic Blockchain Implementation

Introduction

Blockchain is a secure method of storing data in the form of blocks. Every block is connected to the previous block using cryptographic hashes. This makes blockchain very secure because changing one block affects all the blocks after it.

In this task, a simple blockchain was created in Python. Each block contains:

- Index number
- Timestamp
- Transaction data
- Previous block hash
- Current block hash

SHA256 hashing algorithm was used to generate hashes.

Importing Libraries:

```
import hashlib  
import datetime
```

The hashlib library is imported to generate SHA256 hashes.

The datetime library is used to store the current date and time for each block.

Creating the Block Class:

```
class Block:
```

A class named Block is created to represent each block in the blockchain.

Every block stores important information such as transaction data and hashes.

Constructor of Block:

```
def __init__(self, index, timestamp, data, previous_hash):
```

This constructor initializes the values of a block.

- index stores the block number.

- timestamp stores the creation time.
- data stores transaction information.
- previous_hash stores the hash of the previous block.

After storing these values, the current block hash is calculated.

Calculating Hash:

```
self.hash = self.calculate_hash()
```

Each block generates its own hash value.

The hash acts like a digital fingerprint. Even a very small change in data creates a completely different hash.

SHA256 Hash Function:

```
return hashlib.sha256(block_string.encode()).hexdigest()
```

SHA256 converts the block data into a fixed-length secure hash.

This ensures:

- Data integrity
- Security
- Tamper detection

If someone changes transaction data, the hash also changes immediately.

Creating Blockchain Class:

```
class Blockchain:
```

This class manages the entire blockchain.

It is responsible for:

- Creating blocks
- Connecting blocks
- Validating the chain

Genesis Block:

```
create_genesis_block()
```

The first block in blockchain is called the Genesis Block.

It is special because:

- It has no previous block
- Its previous hash is set to 0

Every blockchain always starts with a genesis block.

Adding New Blocks:

`add_block()`

This function adds new blocks into the blockchain.

Before adding:

- The new block receives the hash of the previous block.
- Then its own hash is calculated.

This creates a secure chain connection between blocks.

Displaying Blockchain:

`display_chain()`

This function prints all block information on the screen.

For every block, the following details are displayed:

- Index
- Timestamp
- Data
- Previous hash
- Current hash

Blockchain Validation:

`is_chain_valid()`

This function checks whether the blockchain is secure or not.

It verifies:

1. Whether block hashes are correct

2. Whether previous hashes are properly linked

If someone modifies data in any block, the blockchain becomes invalid.

Demonstrating Blockchain Tampering:

In the program, one block transaction was changed from:

"Ahmed sends 2 BTC to Sara"

to:

"Ahmed sends 100 BTC to Sara"

After changing the data:

- The block hash changed
- Blockchain validation failed

This shows that blockchain can easily detect tampering.

Security Analysis of Blockchain:

Blockchain provides strong security because every block depends on the previous block hash.

Important security features include:

1. Immutability

Once data is added, it cannot be changed easily.

2. Data Integrity

SHA256 hashing protects data from modification.

3. Tamper Detection

Any small modification breaks the blockchain structure.

4. Transparency

Every block can be verified independently.

Because of these features, blockchain is used in:

- Cryptocurrency
- Banking
- Supply chain systems

- Digital transactions

```

BlockChain.py > Block > calculate_hash
1  import hashlib
2  import datetime
3
4  # -----
5  # Block Class
6  # -----
7  class Block:
8
9      def __init__(self, index, timestamp, data, previous_hash):
10
11          self.index = index
12          self.timestamp = timestamp
13          self.data = data
14          self.previous_hash = previous_hash
15
16          # Calculate hash of current block
17          self.hash = self.calculate_hash()
18
19      # Function to calculate SHA256 hash
20      def calculate_hash(self):
21
22          block_string = (
23              str(self.index) +
24              str(self.timestamp) +
25              str(self.data) +
26              str(self.previous_hash)
27          )
28
29          return hashlib.sha256(block_string.encode()).hexdigest()
30
31
32 # -----
33 # Blockchain Class
34 # -----
35 class Blockchain:
36
37     def __init__(self):
38
39         self.chain = [self.create_genesis_block()]
40
41     # Create first block
42     def create_genesis_block(self):
43
44         return Block(
45             0,
46             datetime.datetime.now(),
47             "Genesis Block",
48             "0"
49         )
50
51     # Get latest block
52     def get_latest_block(self):
53
54         return self.chain[-1]
55
56     # Add new block
57     def add_block(self, new_block):
58
59         new_block.previous_hash = self.get_latest_block().hash
60
61         new_block.hash = new_block.calculate_hash()
62

```

```

61         new_block.hash = new_block.calculate_hash()
62
63         self.chain.append(new_block)
64
65     # Display blockchain
66     def display_chain(self):
67
68         for block in self.chain:
69
70             print("\n-----")
71             print("Block Index:", block.index)
72             print("Timestamp:", block.timestamp)
73             print("Data:", block.data)
74             print("Previous Hash:", block.previous_hash)
75             print("Current Hash:", block.hash)
76
77     # Verify blockchain integrity
78     def is_chain_valid(self):
79
80         for i in range(1, len(self.chain)):
81
82             current_block = self.chain[i]
83             previous_block = self.chain[i - 1]
84
85             # Recalculate hash
86             if current_block.hash != current_block.calculate_hash():
87
88                 return False
89
90             # Compare previous hash

```

```

91             if current_block.previous_hash != previous_block.hash:
92
93                 return False
94
95             return True
96
97     # -----
98     # Main Program
99     # -----
100
101     # Create blockchain
102     my_blockchain = Blockchain()
103
104     # Add blocks
105     my_blockchain.add_block(
106         Block(1, datetime.datetime.now(), "Ali sends 5 BTC to Ahmed", "")
107     )
108
109     my_blockchain.add_block(
110         Block(2, datetime.datetime.now(), "Ahmed sends 2 BTC to Sara", "")
111     )
112
113     my_blockchain.add_block(
114         Block(3, datetime.datetime.now(), "Sara sends 1 BTC to Zoya", "")
115     )
116
117     my_blockchain.add_block(
118         Block(4, datetime.datetime.now(), "Zoya sends 3 BTC to Hamza", "")
119     )
120

```

```

121
122 # Display blockchain
123 print("\n===== ORIGINAL BLOCKCHAIN =====")
124 my_blockchain.display_chain()
125
126 # Validate blockchain
127 print("\nIs Blockchain Valid?", my_blockchain.is_chain_valid())
128
129 # -----
130 # Modify one block to show blockchain effect
131 # -----
132
133 print("\n\n===== MODIFYING BLOCK 2 =====")
134
135 my_blockchain.chain[2].data = "Ahmed sends 100 BTC to Sara"
136
137 # Recalculate modified block hash
138 my_blockchain.chain[2].hash = my_blockchain.chain[2].calculate_hash()
139
140 # Display blockchain again
141 my_blockchain.display_chain()
142
143 # Validate again
144 print("\nIs Blockchain Valid After Modification?",
145 | my_blockchain.is_chain_valid())

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\SHOAIB\Desktop\information security> & C:/Users/SHOAIB/AppData/Local
/Programs/Python/Python314/python.exe "c:/Users/SHOAIB/Desktop/information secur
ity/BlockChain.py"

===== ORIGINAL BLOCKCHAIN =====

-----
Block Index: 0
Timestamp: 2026-05-03 18:28:33.726227
Data: Genesis Block
Previous Hash: 0
Current Hash: 3d4925294e3d7e14b1a1cccca5a687df60a6cfb7989e3c5d0fb056d530d7fb8e

-----
Block Index: 1
Timestamp: 2026-05-03 18:28:33.726773
Data: Ali sends 5 BTC to Ahmed
Previous Hash: 3d4925294e3d7e14b1a1cccca5a687df60a6cfb7989e3c5d0fb056d530d7fb8e
Current Hash: 403598ce98a93e606880156be0d870f421515c7d1ac4cd4eee9a9f3c46f54257

-----
Block Index: 2
Timestamp: 2026-05-03 18:28:33.726810
Data: Ahmed sends 2 BTC to Sara
Previous Hash: 403598ce98a93e606880156be0d870f421515c7d1ac4cd4eee9a9f3c46f54257
Current Hash: 9ca33f7fab2fa733266f42b9e9a4a2569795a824bfda66d463065037eb3f3b0c

-----
Block Index: 3
Timestamp: 2026-05-03 18:28:33.726827
Data: Sara sends 1 BTC to Zoya
Previous Hash: 9ca33f7fab2fa733266f42b9e9a4a2569795a824bfda66d463065037eb3f3b0c
Current Hash: 19ffcdce0591f892f8bb9a6933b8b1b5d3f30ea7e9d37491aeb2365aa039c498

```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

-----
Block Index: 4
Timestamp: 2026-05-03 18:28:33.726840
Data: Zoya sends 3 BTC to Hamza
Previous Hash: 19ffcdec0591f892f8bb9a6933b8b1b5d3f30ea7e9d37491aeb2365aa039c498
Current Hash: 2f047aaa2075111f5d1ae0be9f0867027712d08ae23fab7763fda5343e6d422a

Is Blockchain Valid? True

===== MODIFYING BLOCK 2 =====

-----
Block Index: 0
Timestamp: 2026-05-03 18:28:33.726227
Data: Genesis Block
Previous Hash: 0
Current Hash: 3d4925294e3d7e14b1a1cccca5a687df60a6cfb7989e3c5d0fb056d530d7fb8e

-----
Block Index: 1
Timestamp: 2026-05-03 18:28:33.726773
Data: Ali sends 5 BTC to Ahmed
Previous Hash: 3d4925294e3d7e14b1a1cccca5a687df60a6cfb7989e3c5d0fb056d530d7fb8e
Current Hash: 403598ce98a93e606880156be0d870f421515c7d1ac4cd4eee9a9f3c46f54257

-----
Block Index: 2
Timestamp: 2026-05-03 18:28:33.726810
Data: Ahmed sends 100 BTC to Sara
Previous Hash: 403598ce98a93e606880156be0d870f421515c7d1ac4cd4eee9a9f3c46f54257
Current Hash: b687eb357dd260f40c0e9a9f88efed5d2b3f93be8288c52737a58ddc47262fa7
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  +

-----
Block Index: 3
Timestamp: 2026-05-03 18:28:33.726827
Data: Sara sends 1 BTC to Zoya
Previous Hash: 9ca33f7fab2fa733266f42b9e9a4a2569795a824bfda66d463065037eb3f3b0c
Current Hash: 19ffcdec0591f892f8bb9a6933b8b1b5d3f30ea7e9d37491aeb2365aa039c498

-----
Block Index: 4
Timestamp: 2026-05-03 18:28:33.726840
Data: Zoya sends 3 BTC to Hamza
Previous Hash: 19ffcdec0591f892f8bb9a6933b8b1b5d3f30ea7e9d37491aeb2365aa039c498
Current Hash: 2f047aaa2075111f5d1ae0be9f0867027712d08ae23fab7763fda5343e6d422a

Is Blockchain Valid After Modification? False
PS C:\Users\SHOAIB\Desktop\information security>
```

Question no.02:

TASK 2 — Elliptic Curve Cryptography (ECC)

Introduction

Elliptic Curve Cryptography (ECC) is a modern encryption technique used for secure communication.

ECC provides strong security using smaller keys compared to older algorithms like RSA.

It is commonly used in:

- Bitcoin
- SSL certificates
- Secure websites
- Mobile security

Elliptic Curve Equation:

ECC is based on the mathematical equation:

$$y^2 = x^3 + ax + b(\text{mod } p)$$

This equation defines points on an elliptic curve.

Curve Parameters:

$a = 2$

$b = 3$

$p = 97$

These values define the elliptic curve.

- a and b are curve constants
- p is a prime number used for modular arithmetic

Base Point:

$G = (3, 6)$

The point G is called the generator point or base point.

It is used to generate public keys.

Modular Inverse:

ECC calculations require modular division.

Since normal division cannot be used in modular arithmetic, modular inverse is calculated instead.

This function finds a number which satisfies:

$$(a \times a^{-1}) \bmod p = 1$$

Point Addition:

Point addition combines two different points on the curve.

The slope formula used is:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

-10-8-6-4-2246810-10-5510-8.00, -8.008.00, 8.00m = 1.00

Using this slope, a new point is calculated on the curve.

Point Doubling:

Point doubling occurs when the same point is added to itself.

The formula used is:

$$m = \frac{3x_1^2 + a}{2y_1}$$

This operation is very important in ECC key generation.

Scalar Multiplication:

Scalar multiplication means repeatedly adding the same point.

For example:

$5 \times G$

This operation generates the public key.

Key Generation:

ECC uses:

- Private key
- Public key

The private key is kept secret.

The public key is generated using scalar multiplication.

Example:

private_key = 5

public_key = 5 × G

Encryption Process:

During encryption:

- A random value k is selected
- Ciphertext values C1 and C2 are generated

This protects the original message.

Decryption Process:

The receiver uses the private key to recover the original message.

Only the correct private key can decrypt the message successfully.

Security Analysis of ECC:

ECC is considered highly secure because it is based on a difficult mathematical problem called:

Elliptic Curve Discrete Logarithm Problem (ECDLP)

This problem is extremely difficult to solve.

Main advantages of ECC include:

1. Strong Security

Provides excellent protection against attacks.

2. Smaller Key Size

Uses shorter keys while maintaining high security.

3. Faster Performance

Requires less processing power.

4. Widely Used

Used in cryptocurrencies and secure internet communication.

```

ecc.py > ...
1  # -----
2  # ECC BASIC IMPLEMENTATION
3  # -----
4
5  # Elliptic Curve:
6  #  $y^2 = x^3 + ax + b \pmod p$ 
7
8  a = 2
9  b = 3
10 p = 97
11
12 # Base point G
13 G = (3, 6)
14
15 # -----
16 # Modular Inverse Function
17 # -----
18
19 def mod_inverse(k, p):
20     for i in range(1, p):
21         if (k * i) % p == 1:
22             return i
23     return None
24
25 # -----
26 # Point Addition Function
27 # -----

```

```

def mod_inverse(k, p):
    for i in range(1, p):
        if (k * i) % p == 1:
            return i
    return None

```

```

ecc.py > ...
33
34 def point_addition(P, Q):
35
36     if P == Q:
37         return point_doubling(P)
38
39     x1, y1 = P
40     x2, y2 = Q
41
42     m = ((y2 - y1) * mod_inverse(x2 - x1, p)) % p
43
44     x3 = (m * m - x1 - x2) % p
45     y3 = (m * (x1 - x3) - y1) % p
46
47     return (x3, y3)
48
49
50 # -----
51 # Point Doubling Function
52 # -----
53
54 def point_doubling(P):
55
56     x1, y1 = P
57
58     m = ((3 * x1 * x1 + a) *
59          mod_inverse(2 * y1, p)) % p
60
61     x3 = (m * m - 2 * x1) % p
62     y3 = (m * (x1 - x3) - y1) % p
63
64     return (x3, y3)

```

```

def point_addition(P, Q):
    if P == Q:
        return point_doubling(P)
    x1, y1 = P
    x2, y2 = Q
    m = ((y2 - y1) * mod_inverse(x2 - x1, p)) % p
    x3 = (m * m - x1 - x2) % p
    y3 = (m * (x1 - x3) - y1) % p
    return (x3, y3)

```

```

ecc.py > ...
65
66
67 # -----
68 # Scalar Multiplication
69 # -----
70
71 def scalar_multiplication(k, P):
72     result = P
73
74     for i in range(k - 1):
75         result = point_addition(result, P)
76
77     return result
78
79
80
81 # -----
82 # Key Generation
83 # -----
84
85 private_key = 5
86
87 public_key = scalar_multiplication(private_key, G)
88
89 print("\nPrivate Key:", private_key)
90 print("Public Key:", public_key)
91
92 # -----
93 # Encryption
94 # -----
95
96

```

```

ecc.py > ...
97 message = 10
98
99 k = 2
100
101 C1 = scalar_multiplication(k, G)
102
103 shared_secret = scalar_multiplication(k, public_key)
104
105 C2 = message + shared_secret[0]
106
107 print("\nEncrypted Message:")
108 print("C1 =", C1)
109 print("C2 =", C2)
110
111 # -----
112 # Decryption
113 # -----
114
115 decryption_secret = scalar_multiplication(private_key, C1)
116
117 decrypted_message = C2 - decryption_secret[0]
118
119 print("\nDecrypted Message:", decrypted_message)

```

Output:

o 02.py Question no03.py BlockChain.py task ecc.py

ecc.py > ...
101 C1 = scalar_multiplication(k, G)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\SHOAIB\Desktop\information security> & C:/Users/SHOAIB/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/SHOAIB/Desktop/information security/ecc.py"
Traceback (most recent call last):
 File "c:\Users\SHOAIB\Desktop\information security\ecc.py", line 88, in <module>
 public_key = scalar_multiplication(private_key, G)
 File "c:\Users\SHOAIB\Desktop\information security\ecc.py", line 77, in scalar_multiplication
 result = point_addition(result, P)
 File "c:\Users\SHOAIB\Desktop\information security\ecc.py", line 42, in point_addition
 m = ((y2 - y1) * mod_inverse(x2 - x1, p)) % p
~~~~~~^~~~~~  
TypeError: unsupported operand type(s) for \*: 'int' and 'NoneType'  
PS C:\Users\SHOAIB\Desktop\information security>